

Study Guide

This study guide provides a comprehensive overview of Exception Handling as presented in the Karnataka 2nd PUC (Class 12) Computer Science syllabus (Subject Code 41). It follows the NCERT/CBSE Class 12 Python curriculum, focusing on how programs manage runtime issues to ensure robustness and reliability.

I. Core Concepts Summary

Understanding Errors

Python programs encounter two primary types of issues that prevent successful execution:

- Syntax Errors (Parsing Errors):** These are mistakes in the “grammar” of the code, such as a missing colon after an `if` statement. They are detected by Python before the program begins to run. A program will not execute until all syntax errors are corrected.
- Exceptions (Runtime Errors):** These occur when the code is grammatically correct and has begun execution, but an error arises during the process. When Python detects such a problem, it “raises” an exception. If the exception is not handled, the program terminates abruptly and displays a traceback.

The Exception Mechanism

Exceptions in Python are objects belonging to classes that inherit from a base class named `Exception`. While programmers can define their own, Python includes several “built-in” exceptions to handle common scenarios.

The try...except Block Structure

To prevent programs from crashing, Python uses a structured approach to catch and manage errors:

Block	Function
<code>try</code>	Contains the “risky” code that might trigger an error.
<code>except</code>	Executes only if a matching exception occurs within the <code>try</code> block.

Block	Function
<code>else</code>	Executes only if the <code>try</code> block completes successfully without any exceptions.
<code>finally</code>	Always executes, regardless of whether an exception occurred. It is typically used for “clean-up” tasks, such as closing files.

II. Short-Answer Practice Questions

- 1. What is the primary difference between a syntax error and an exception?** A syntax error is a grammatical mistake detected before the program runs (parsing stage), preventing it from starting. An exception is a runtime error that occurs while the program is already executing, even if the code is grammatically valid.
- 2. What happens to a program if an exception is raised but not handled?** The program will stop execution immediately and print a “traceback,” which is a report of the error.
- 3. Why is it considered a poor programming practice to use a “bare” `except` block?** A bare `except` block is discouraged because it catches every possible error indiscriminately, which can hide underlying bugs and make debugging difficult.
- 4. How can a programmer capture the specific exception object for inspection?** A programmer can use the `as` keyword. For example: `except ZeroDivisionError as e`.
- 5. How are multiple exceptions handled in a single `try` block?** A single `try` block can have multiple `except` blocks which Python checks from top to bottom. Alternatively, several exceptions can be grouped together in a tuple, such as `except (ValueError, TypeError):`.
- 6. What is the purpose of the `raise` statement?** The `raise` statement allows a programmer to deliberately trigger a specific exception when a certain condition is met (e.g., `raise ValueError("Age cannot be negative!")`).

III. Essay Questions for Deeper Exploration

- 1. Discuss the Significance of Exception Handling in Software Development.** *Analysis Guide:* Your response should explain how exception handling prevents programs from crashing and makes them more “robust.” Discuss how it replaces confusing technical tracebacks with user-friendly messages and separates normal program logic from error-handling logic. Mention its role

in guaranteeing resource management (like closing files) through the `finally` block, particularly when dealing with unpredictable factors like user input or external files.

2. Compare and Contrast the Roles of the `else` and `finally` Blocks. *Analysis Guide:*

Explain that while both blocks follow `try` and `except`, they serve different triggers. The `else` block is conditional—it only runs if the `try` block succeeds without error. In contrast, the `finally` block is mandatory—it runs whether an error occurred or not. Use examples such as using `else` for logic that should only happen on success, and `finally` for essential clean-up tasks like releasing system resources.

IV. Glossary of Key Terms

Term	Definition
Traceback	The error report printed by Python when an unhandled exception causes a program to stop.
Built-in Exception	Pre-defined exception classes already existing in Python (e.g., <code>NameError</code>).
ZeroDivisionError	Raised when the program attempts to divide a number by zero.
ValueError	Raised when a function receives an argument of the correct type but an invalid value (e.g., <code>int("abc")</code>).
TypeError	Raised when an operation is applied to an object of an inappropriate type (e.g., adding a string to an integer).
NameError	Raised when the program tries to access a variable name that has not been defined.
IndexError	Raised when attempting to access a list index that is out of the valid range.
KeyError	Raised when a program attempts to access a dictionary using a key that does not exist.
FileNotFoundError	Raised when the program attempts to open or move a file that does not exist in the specified directory.

Term	Definition
Clean-up	The process of releasing resources (like closing files) performed in the <code>finally</code> block to ensure system stability.